

ENGINEERING BUILD SPECIFICATION

Ethiopia Accountability, Learning & Community Platform

Document type: Executable product + engineering specification

Audience: Senior developers, AI coding agents, designers, QA engineers, security engineers and future maintainers

Initial platform: Responsive Web + PWA

Backend: Supabase/PostgreSQL

Frontend: Next.js + React + TypeScript

Design: Minimal, premium, restrained, light/dark

Initial cost target: \$0

Launch strategy: Closed beta → controlled public launch → national growth

1. WHAT IS ACTUALLY BEING BUILT?

Build a social accountability platform for Ethiopia.

A user should be able to:

1. create an account
2. describe their interests
3. discover communities
4. join communities
5. communicate with community members
6. follow people
7. create personal goals
8. turn goals into routines
9. check in on completed routines
10. optionally attach evidence
11. invite accountability partners
12. participate in community challenges
13. see progress
14. receive useful reminders
15. discover people with similar interests
16. send direct messages
17. report/block users
18. participate in moderated communities

Community administrators should be able to:

1. create/manage communities
2. moderate members

3. create announcements
4. create challenges
5. manage community content
6. appoint moderators

Platform administrators should be able to:

1. manage users
2. manage communities
3. manage moderators
4. review reports
5. remove content
6. suspend/ban users
7. review advertisements
8. inspect platform metrics
9. maintain platform safety

2. WHAT THE FIRST USER SHOULD EXPERIENCE

The first-time experience is:

```
Landing page
  ↓
Create account
  ↓
Tell us about yourself
  ↓
Choose interests
  ↓
Choose experience levels
  ↓
Choose what you want from the platform
  ↓
Recommended communities
  ↓
Join communities
  ↓
Home
```

The user must never land on an empty application.

3. GLOBAL FRONTEND REQUIREMENTS

Every page must support:

```
Desktop
Tablet
Mobile
Light mode
Dark mode
Loading
Empty
Error
Offline/degraded connection
```

The mobile experience takes priority.

Minimum supported layout:

```
360px width
```

No core functionality may depend on hover.

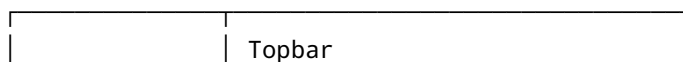
4. GLOBAL APPLICATION SHELL

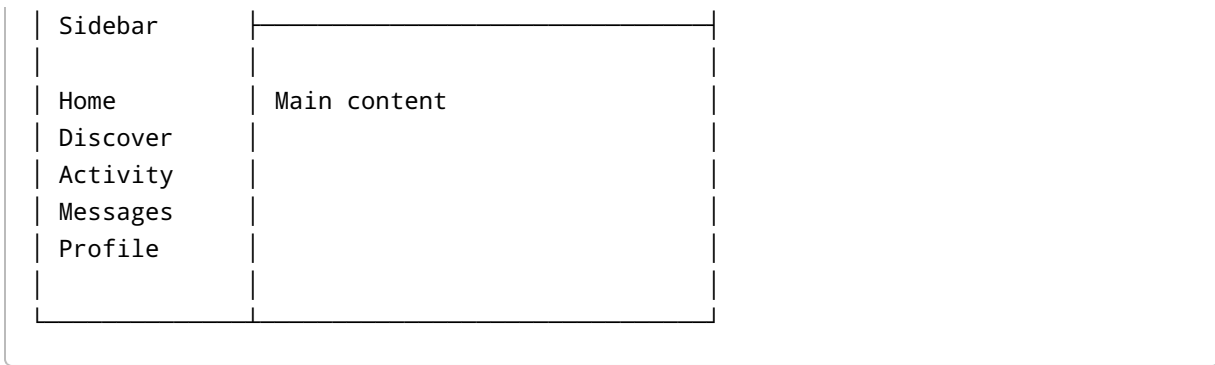
Build

Create:

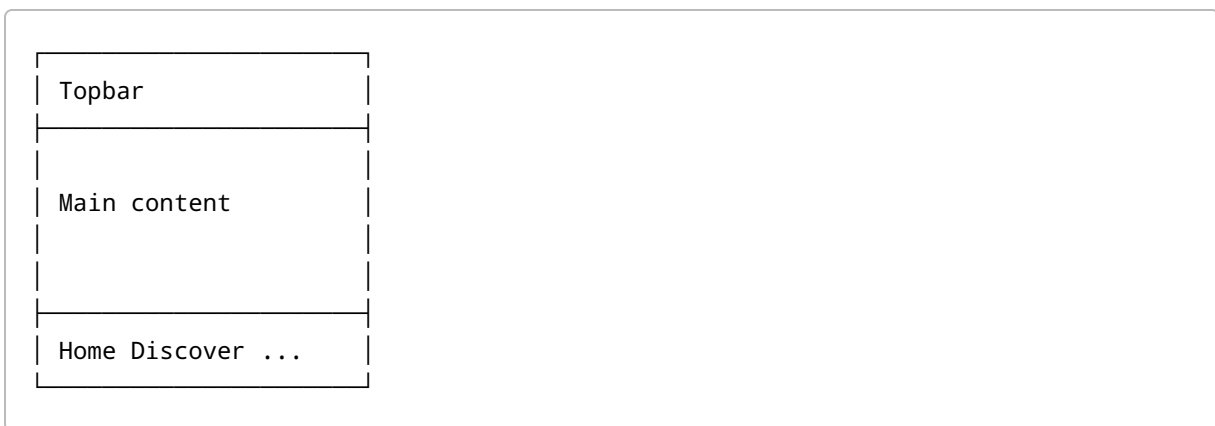
```
AppShell
├─ DesktopSidebar
├─ MobileBottomNavigation
├─ TopBar
├─ MainContent
└─ GlobalNotificationLayer
```

Desktop:





Mobile:



5. DESIGN SYSTEM

Create reusable components before building product pages.

Required components

Button
IconButton
Input
Textarea
Select
Checkbox
Radio
Switch
Avatar
Badge
Card
Dialog
Drawer

- Dropdown
- Tabs
- Toast
- Tooltip
- Skeleton
- ProgressBar
- ProgressRing
- EmptyState
- ErrorState
- ConfirmDialog
- Pagination
- SearchInput
- FileUploader

Required variants

Buttons:

- primary
- secondary
- outline
- ghost
- danger

Every component must work in:

- light
- dark
- mobile
- desktop

6. COLOR SYSTEM

Do not hardcode colors throughout the application.

Use semantic tokens.

Light:

- background
- surface

```
surface-secondary  
border  
text-primary  
text-secondary  
text-muted  
accent  
success  
warning  
danger
```

Dark has separate values.

The visual style should be:

- neutral
- restrained
- premium
- readable
- low visual noise

Avoid:

- neon
- excessive gradients
- glowing effects
- excessive glass effects
- excessive shadows
- unnecessary animations

7. DATABASE — CORE TABLES

Create migrations for:

```
profiles  
interests  
user_interests  
  
communities  
community_members  
community_roles  
  
posts  
comments  
reactions
```

```
goals
goal_routines
goal_checkins
goal_evidence

challenges
challenge_participants
challenge_checkins

follows

accountability_partners

conversations
conversation_members
messages

notifications
notification_preferences

reports
moderation_actions
blocks

reputation_events

media

advertisements
advertisement_reviews

audit_logs
```

8. USER PROFILE

Database

profiles

Required fields:

```
id
username
display_name
bio
avatar_url
location_region      optional
experience_summary    optional
created_at
updated_at
```

Never store sensitive personal information unless genuinely necessary.

9. PROFILE PAGE

Route:

```
/profile/[username]
```

Build:

```
ProfileHeader
Avatar
Display name
Username
Bio
Follow button
Message button
Interests
Current goals
Communities
Achievements
Reputation
```

User actions

```
Follow
Unfollow
Message
Block
Report
```


Privacy

Users must control visibility of:

```
goals
activity
communities
profile
```

10. ONBOARDING

Route:

```
/onboarding
```

Screen 1

Ask:

```
What should we call you?
```

Fields:

```
display name
username
```

Validate username for:

- uniqueness
 - allowed characters
 - length
 - reserved words
-

Screen 2 — Interests

Display categories and searchable interests.

Example:

Programming
Cybersecurity
Forex
Fitness
Calisthenics
Bible study
Business
Reading
Football
Languages
Music
Art

User can select multiple.

Store in:

user_interests

Screen 3 — Experience

For each selected interest:

Beginner
Learning
Intermediate
Advanced
Professional/Experienced

Screen 4 — Intent

Ask:

What are you here for?

Allow:

Accountability
Learning
Finding people
Helping others
Challenges
Discussion
All of these

Screen 5 — Recommendations

Query communities matching:

user interests
experience
intent
location where voluntarily supplied

Display community cards.

User can:

Join
Skip

11. HOME

Route:

/home

The home screen is the user's daily command center.

Components

Greeting
TodayAccountability
GoalSummary

UpcomingDeadline
PartnerActivity
CommunityActivity
RecommendedAction

Example:

TODAY

JavaScript
1 hour
[Start]

Workout
30 minutes
[Complete]

Reading
20 pages
[Complete]

Below:

COMMUNITY ACTIVITY

Then:

YOUR NEXT DEADLINE

Then:

PEOPLE YOU MAY KNOW

12. GOALS

Route:

/goals

Users can:

```
create
view
edit
pause
complete
archive
delete
```

13. CREATE GOAL

Build a multi-step form.

Fields:

```
title
description
category
start_date
target_date
visibility
```

Optional:

```
measurement
target_value
unit
```

Example:

```
Learn JavaScript

Target:
Complete JavaScript fundamentals

Deadline:
October 30
```

14. GOAL DETAIL

Route:

```
/goals/[id]
```

Display:

```
Goal title
Description
Deadline
Progress
Routine
Streak
Check-in history
Evidence
Accountability partner
```

Actions:

```
Check in
Edit
Pause
Complete
Archive
```

15. ROUTINES

A goal may contain one or more routines.

Example:

```
Goal:
Learn JavaScript

Routine:
Study 60 minutes
```

Frequency:
Every weekday

Supported frequencies:

daily
weekly
specific weekdays
custom interval

16. CHECK-IN SYSTEM

This is one of the most important systems.

User action

Press:

Complete

Open lightweight check-in interface.

Fields:

completion status
optional note
optional evidence

Completion should require minimal interaction.

17. CHECK-IN DATABASE

goal_checkins

Store:

```
id
goal_id
user_id
routine_id
scheduled_date
completed_at
status
note
created_at
```

Possible status:

```
completed
skipped
missed
```

Prevent duplicate check-ins for the same routine/date.

Database constraints must enforce this.

18. STREAK CALCULATION

Do not blindly store a streak number and trust it.

Calculate streaks from actual check-in records.

Support:

```
current streak
longest streak
completion rate
```

Define exactly how missed days affect streaks.

Document this algorithm.

19. EVIDENCE

Allow optional evidence.

MVP:

```
image
text
```

Later:

```
documents
additional media
```

Upload flow:

```
Select file
↓
Validate type
↓
Validate size
↓
Upload
↓
Associate with check-in
```

Never expose private storage objects directly.

Use appropriate access policies/signed URLs.

20. ACCOUNTABILITY PARTNER

Users can invite another user to become a partner.

Flow:

```
User A
↓
Invite User B
↓
User B accepts
↓
Partnership active
```

Partner can see only information explicitly shared.

Partner features:

- view shared goal
- see check-in status
- send encouragement
- receive relevant reminder

Partner cannot modify another person's goal.

21. DISCOVER

Route:

/discover

Build:

- Search
- Category navigation
- Recommended communities
- Popular communities
- New communities
- Active communities

Community card:

- name
- description
- category
- member count
- activity indicator
- verification indicator
- join button

22. COMMUNITY PAGE

Route:

```
/communities/[slug]
```

Build:

```
CommunityHeader  
CommunityNavigation  
CommunityFeed  
CommunityChallenges  
CommunityResources  
CommunityMembers
```

Tabs:

```
Overview  
Posts  
Challenges  
Resources  
Members
```

23. COMMUNITY CREATION

Initially restrict community creation.

Potential MVP model:

```
User requests community  
↓  
Admin reviews  
↓  
Approved  
↓  
Community created
```

Later allow trusted users to create communities automatically with moderation controls.

24. COMMUNITY ROLES

Roles:

```
member
moderator
admin
owner
```

Platform-level:

```
platform_moderator
platform_admin
```

Permissions must be database-enforced.

25. COMMUNITY POSTS

Users can create:

```
text post
image post
```

Fields:

```
title
body
media
community
author
```

Actions:

```
edit own
delete own
comment
react
report
```

26. COMMUNITY ANNOUNCEMENTS

Community admins can create announcements.

Announcements appear prominently.

Examples:

```
Challenge starting Monday
Community rules
Important event
Weekly discussion
```

Members cannot create announcements.

27. COMMENTS

Comments support:

```
create
edit own
delete own
react
report
```

Avoid unlimited nested comment trees initially.

Use:

```
post
└─ comments
```

instead of Reddit-style infinite nesting.

28. REACTIONS

Start with a limited reaction system.

For example:

```
Like
Helpful
Support
```

Do not build dozens of reactions.

29. CHALLENGES

Route:

```
/challenges
/challenges/[id]
```

A challenge contains:

```
name
description
community_id
creator_id
start_date
end_date
requirements
rules
```

Example:

```
30 Days of Coding

Complete at least
30 coding sessions
within 30 days.
```

30. CHALLENGE PARTICIPATION

User:

```
Join challenge
↓
Complete requirements
↓
Check in
↓
Progress updates
↓
Challenge completed
```

Participant progress is visible according to challenge privacy settings.

31. MESSAGING

Route:

```
/messages
/messages/[conversationId]
```

MVP supports:

```
one-to-one conversations
text messages
message timestamps
read state
block
report
```

Do not build:

```
voice
video
file sharing
GIF system
complex reactions
```

until later.

32. MESSAGE SECURITY

Before sending a message:

```
authenticate user
verify conversation membership
validate content
rate limit
check block relationship
```

The database must prevent a user from reading conversations they do not belong to.

33. FOLLOW SYSTEM

Users can:

```
follow
unfollow
```

Do not automatically make following expose private accountability information.

Following affects:

```
recommendations
activity
social discovery
```

according to privacy settings.

34. NOTIFICATIONS

Build a notification service.

Events include:

```
goal reminder
deadline approaching
partner check-in
```



```
challenge update
community announcement
comment
reaction
follow
message
moderation action
```

Store notifications in:

```
notifications
```

User settings in:

```
notification_preferences
```

35. REMINDERS

Users should be able to configure:

```
notification time
frequency
enabled/disabled
```

Do not spam.

Default notification behavior should be conservative.

36. ACTIVITY

Route:

```
/activity
```

Show meaningful activity:

```
goal completed
challenge joined
community activity
partner activity
achievements
```

Do not create a feed consisting entirely of meaningless micro-actions.

37. SEARCH

Initial search:

```
communities
users
posts
```

Search must:

- normalize input
 - rate-limit requests
 - paginate results
 - handle no results
 - protect private profiles/content
-

38. REPORTING

Any relevant user-generated content should expose:

```
Report t
```

Report categories:

```
Spam
Harassment
Scam
Illegal content
NSFW
Dangerous misinformation
```

Impersonation
Other

Create:

reports

with:

reporter
target
target_type
reason
description
status
created_at
resolved_at
resolved_by

39. BLOCKING

Users can block other users.

Blocking should prevent appropriate interactions such as:

messages
direct interaction
certain profile discovery

Define exact behavior in the security specification.

40. MODERATION DASHBOARD

Route:

/admin/moderation

Display:

```
Pending reports
High-priority reports
Recent actions
Banned users
Flagged content
```

Moderator can:

```
review
remove content
warn user
restrict user
suspend user
ban user
dismiss report
```

Every action creates an audit record.

41. ADMIN DASHBOARD

Route:

```
/admin
```

Dashboard metrics:

```
total users
new users
active users
communities
posts
goals
check-ins
reports
pending moderation
```

Do not build complex analytics initially.

42. ADVERTISEMENT SYSTEM

Do not implement monetization in MVP.

Design database interfaces so it can later support:

```
advertisements
advertisement_reviews
advertisement_campaigns
```

Future flow:

```
Advertiser
↓
Submit
↓
Admin review
↓
Approve/reject
↓
Target community/category
↓
Publish
```

43. SUPPORT COMMUNITIES

Support/recovery communities need special configuration.

Community settings should eventually include:

```
anonymous participation
restricted discoverability
extra moderation
content warnings
support resources
```

Peer support must never be represented as professional treatment.

44. FINANCIAL COMMUNITIES

For Forex/trading communities:

The platform provides:

discussion
accountability
education
journaling
community

It should not represent:

guaranteed returns
guaranteed strategies
financial certainty

Add appropriate community rules/disclaimers.

45. REPUTATION

Create an event-based reputation system.

Example events:

helpful contribution
challenge completion
community contribution
moderator recognition

Negative events:

confirmed abuse
spam
moderation violation

Do not let users directly manipulate reputation.

46. MODERATION + REPUTATION

Reputation should influence:

```
community discovery
creator permissions
community creation eligibility
moderator eligibility
```

But never allow reputation to bypass core safety rules.

47. PWA

Configure:

```
manifest
icons
service worker
installability
offline shell
cache strategy
```

The application should remain usable during temporary connection failures.

48. OFFLINE CHECK-IN

Important functionality:

If user checks in while temporarily offline:

```
UI
↓
Local queue
↓
"Saved locally"
↓
Connection restored
↓
Synchronize
```

↓

Server confirmation

The server remains authoritative.

Use idempotency keys to prevent duplicate submissions.

49. LOW-BANDWIDTH MODE

Implement an application strategy that minimizes:

image sizes
JavaScript
unnecessary requests
repeated downloads
large feed payloads

Future setting:

Data Saver

When enabled:

- lower image quality
 - lazy-load media
 - avoid unnecessary prefetching
-

50. SECURITY ARCHITECTURE

Every server operation must perform:

Authentication
Authorization
Validation
Business rule validation
Rate limiting where necessary
Database operation
Audit logging where necessary

Never trust the browser.

51. API/SERVER CONTRACT

For every operation define:

```
Input
Authentication requirement
Authorization requirement
Validation
Success response
Failure response
Side effects
Database operations
```

Example:

```
createGoal()

Input:
title
description
deadline
visibility

Requires:
authenticated user

Can create:
only for current user

Returns:
goal

Errors:
invalid input
deadline invalid
rate limited
```

Do this for every server operation.

52. VALIDATION

Use a shared validation layer.

Frontend validation improves UX.

Backend validation provides security.

Both must exist.

53. ERROR HANDLING

Never expose raw database errors to users.

Bad:

```
PostgreSQLError: duplicate key...
```

Good:

```
That username is already taken.
```

Log technical details privately.

54. LOGGING

Production logs should contain enough information to diagnose:

```
request
user
operation
error
timestamp
environment
```

Never log:

```
passwords
tokens
private messages
sensitive personal information
```

55. RATE LIMITING

Apply limits to:

```
login attempts
signup
password recovery
messages
posts
comments
reports
uploads
search
community creation
```

Exact limits should be configurable rather than hardcoded throughout the application.

56. FILE UPLOAD SECURITY

For every upload:

```
check MIME type
check extension
check file size
generate safe filename
store outside executable paths
restrict access
```

Images should be transformed into safe formats where appropriate.

57. TESTING REQUIREMENTS

Every feature must include tests.

Example goal system:

```
create goal
invalid goal
edit own goal
cannot edit another user's goal
delete own goal
cannot delete another user's goal
create routine
complete routine
duplicate check-in
missed check-in
streak calculation
timezone edge cases
```

58. END-TO-END TESTS

Automate:

Signup

```
open site
create account
complete onboarding
reach home
```

Community

```
search
open community
join
create post
comment
```

Accountability

```
create goal  
create routine  
check in  
verify progress
```

Messaging

```
open conversation  
send message  
receive message
```

Moderation

```
report content  
moderator reviews  
moderator action  
user sees appropriate result
```

59. TIMEZONE HANDLING

This must be designed early.

Store timestamps in UTC.

Display dates according to the user's configured timezone.

Goal/routine calculations must explicitly define what constitutes a "day."

Do not let streak logic accidentally use server timezone.

60. DATA OWNERSHIP

Users can manage their own:

```
profile  
goals
```

```
routines
check-ins
messages where applicable
posts
```

Community admins manage community resources.

Platform admins manage platform-level moderation.

61. ACCOUNT DELETION

Implement account deletion infrastructure early.

Flow:

```
User requests deletion
↓
Confirmation
↓
Account deactivation/deletion process
↓
Personal data removed/anonymized according to retention rules
```

Define what must remain for:

```
security
moderation
legal obligations
audit history
```

before implementation.

62. DATA EXPORT

Not necessarily exposed in the MVP UI, but architecture should avoid making user data impossible to export.

Future:

Download my data

63. PRIVACY

Privacy settings should eventually include:

```
profile visibility
goal visibility
activity visibility
online status
message permissions
```

Default settings should favor reasonable privacy.

64. RECOMMENDATION SYSTEM — MVP

Do NOT build machine-learning recommendations.

Use deterministic scoring.

Example:

```
interest match
+
community activity
+
experience compatibility
+
existing membership
+
language/location relevance
```

This is cheaper, explainable and sufficient initially.

65. FUTURE RECOMMENDATION ENGINE

Only after sufficient usage data:

```
user behavior
community behavior
goal similarity
content interaction
successful matches
```

Then evaluate more sophisticated recommendation systems.

Do not add generative AI merely because AI is fashionable.

66. AI CONTENT MODERATION

AI can eventually assist with:

```
spam classification
duplicate detection
toxicity detection
NSFW detection
scam patterns
```

But high-impact moderation decisions should have human oversight.

For the zero-budget MVP, begin with:

```
rules
rate limits
reports
moderators
```

67. FRONTEND PAGE INVENTORY

The developer must build these screens.

Public

```
Landing
Login
```


Signup
Forgot Password

Onboarding

Welcome
Profile
Interests
Experience
Intent
Community Recommendations

Main

Home
Discover
Activity
Messages
Profile
Settings

Community

Community
Community Posts
Community Challenges
Community Resources
Community Members

Accountability

Goals
Create Goal
Goal Detail
Edit Goal
Check-in
Challenge

Social

User Profile
Followers/Following
Conversation

Safety

Report dialog
Blocked users
Moderation notification

Admin

Admin Dashboard
User Management
Community Management
Reports
Moderation
Advertisement Review

68. EXACT MVP NAVIGATION

Desktop:

Home
Discover
Activity
Messages

Your communities
 Python Ethiopia
 Fitness Ethiopia
 ...

Profile
Settings

Mobile:

Home
Discover
Activity
Messages
Profile

Community navigation occurs inside community pages.

69. HOME PRIORITY

The home screen must NOT become:

random social feed
random advertisements
viral posts

Priority:

1. Today's goals
2. Important accountability
3. Upcoming deadlines
4. Partner activity
5. Community updates
6. Discovery

70. DISCOVER PRIORITY

Discovery should help answer:

"Where are the people interested in what I care about?"

Show:

Recommended
Trending/Active
New

Categories
Search

71. EMPTY STATES

Every page must have intentional empty states.

Example:

You haven't joined any communities yet.

Find people working on the
same things you are.

[Discover communities]

Never show a blank page.

72. LOADING STATES

Use skeletons for content-heavy screens.

Do not show a spinner for everything.

Buttons should show localized loading:

Joining...

Saving...

Sending...

73. ERROR STATES

Example:

Couldn't load this community.

[Try again]

Do not force users to refresh the whole page.

74. RESPONSIVE BREAKPOINTS

Use the design system's breakpoints consistently.

Do not write arbitrary breakpoints inside individual components.

75. ANIMATION

Animation should communicate:

state change
navigation
completion
feedback

Keep it fast and subtle.

Respect:

prefers-reduced-motion

76. PERFORMANCE BUDGET

Establish measurable budgets.

Monitor:

initial page load
JavaScript size
image transfer

```
API response time
database query time
interaction latency
```

Set targets during implementation based on actual devices and network testing.

77. OBSERVABILITY

Eventually include:

```
error monitoring
performance monitoring
uptime monitoring
database monitoring
```

For a zero-budget MVP, use free tiers where available.

78. DEPLOYMENT

Production architecture:

```
GitHub
↓
CI
↓
Preview deployment
↓
Production deployment
```

Database migrations run through controlled deployment procedures.

79. ENVIRONMENT VARIABLES

Separate:

```
local
preview
production
```

Never commit secrets.

Provide:

```
.env.example
```

with variable names but no secrets.

80. PROJECT DOCUMENTATION

Repository must contain:

```
README.md
AGENTS.md

docs/
├─ ARCHITECTURE.md
├─ DATABASE.md
├─ SECURITY.md
├─ FRONTEND.md
├─ PRODUCT.md
├─ MODERATION.md
├─ DEPLOYMENT.md
├─ TESTING.md
├─ PROJECT_STATUS.md
├─ DECISIONS/
└─ FEATURES/
```

81. AGENTS.MD

This file must tell every AI coding agent:

```
You are working on...
```

Tech stack:

...

Never:

...

Always:

...

Database rules:

...

Security rules:

...

Frontend rules:

...

Testing requirements:

...

Before modifying schema:

...

Before adding dependencies:

...

After completing a feature:

...

82. FEATURE DEVELOPMENT PROTOCOL

No developer should simply receive:

"Build messaging."

Instead they receive:

FEATURE: Direct Messaging

Goal:

Allow users to communicate privately.

Dependencies:


```
Authentication
Profiles
Blocks

Build:
Conversation list
Conversation page
Message composer
Send message
Read state
Block behavior
Report behavior

Database:
conversations
conversation_members
messages

Security:
Only members can read/send.

Tests:
...
```

Every feature specification should follow this structure.

83. FEATURE COMPLETION CHECKLIST

Before marking a feature complete:

```
[ ] Requirements implemented
[ ] Database migration
[ ] RLS/security
[ ] Backend/server logic
[ ] Validation
[ ] Frontend
[ ] Mobile
[ ] Light mode
[ ] Dark mode
[ ] Loading state
[ ] Empty state
[ ] Error state
[ ] Accessibility
[ ] Tests
```

- [] Security review
- [] Documentation
- [] Analytics event if required
- [] Performance check

84. SENIOR DEVELOPER OPERATING INSTRUCTIONS

Before writing code:

1. Read `README.md`.
2. Read `AGENTS.md`.
3. Read `PROJECT_STATUS.md`.
4. Read relevant feature specification.
5. Inspect existing architecture.
6. Inspect existing database migrations.
7. Identify dependencies.
8. Propose implementation plan.
9. Identify security risks.
10. Only then modify code.

Never rewrite functioning architecture simply because another approach looks cleaner.

85. DATABASE CHANGE OPERATING INSTRUCTIONS

Before modifying the database:

1. Inspect existing schema.
2. Determine whether the feature actually needs new tables.
3. Design relationships.
4. Define constraints.
5. Define indexes.
6. Define RLS policies.
7. Create migration.
8. Test migration.
9. Update database documentation.

Never make undocumented production schema changes.

86. FRONTEND CHANGE OPERATING INSTRUCTIONS

Before creating a component:

1. Search existing components.
2. Reuse existing primitives.
3. Follow design tokens.
4. Check mobile behavior.
5. Check dark mode.
6. Implement loading/error/empty states.
7. Test keyboard accessibility.

Do not create five visually different buttons for five pages.

87. CODE REVIEW PROMPT

After implementation, the AI/developer should run this review:

Review this feature as a senior production engineer. Assume the client is malicious and the network is unreliable. Find authorization flaws, data leaks, race conditions, duplicate operations, validation failures, performance issues, accessibility problems, mobile UI failures, state-management problems and maintainability issues. Return findings by severity and provide exact fixes.

88. SECURITY REVIEW PROMPT

Assume the attacker controls the browser, request payloads, IDs and timing. Attempt to access another user's data, modify another user's resources, bypass community permissions, bypass moderation, spam the system, upload malicious files, manipulate reputation and exploit race conditions.

89. TEST GENERATION PROMPT

Generate unit, integration and end-to-end tests for this feature. Include normal behavior, invalid inputs, authorization failures, concurrency, duplicate requests, offline behavior, mobile behavior and regression cases.

90. DOCUMENTATION PROMPT

Explain this feature for a developer who has never seen the project. Explain its purpose, user flow, architecture, database relationships, permissions, important implementation details, failure modes, security assumptions and how to safely modify it.

91. HANDOFF REQUIREMENT

When a developer stops working:

They must update:

```
PROJECT_STATUS.md
feature status
known bugs
migration status
unfinished work
next task
architectural concerns
```

Then commit everything.

The next developer should be able to clone the repository and determine:

```
Where are we?
What works?
What doesn't?
What is next?
Why was it built this way?
```

without asking the previous developer.

92. MVP ACCEPTANCE TEST

The MVP is not considered ready until a new user can complete this entire journey:

```
Create account
↓
Complete onboarding
```

↓
See relevant communities
↓
Join community
↓
Read community post
↓
Create personal goal
↓
Create routine
↓
Complete check-in
↓
See progress
↓
Find another user
↓
Follow them
↓
Send message
↓
Invite accountability partner
↓
Return later
↓
Receive relevant reminder
↓
Complete another check-in

A moderator must simultaneously be able to:

see reports
review content
take action
see audit history

93. FIRST DEVELOPMENT SPRINT

Do not start by building the entire application.

Build exactly this:

```
01 Repository
02 Next.js application
03 TypeScript
04 Tailwind
05 Design tokens
06 Component primitives
07 Supabase
08 Authentication
09 Profiles
10 Onboarding
11 Interests
12 App shell
13 Home skeleton
14 Discover skeleton
15 Community database
16 Community discovery
17 Community detail
18 Join community
```

Stop and test.

Then proceed to accountability.

94. SECOND DEVELOPMENT SPRINT

```
01 Goals database
02 Goal creation
03 Goal detail
04 Routine creation
05 Check-in
06 Progress calculation
07 Streak calculation
08 Evidence upload
09 Goal dashboard
10 Home accountability module
```

95. THIRD DEVELOPMENT SPRINT

```
01 Posts
02 Comments
```

- 03 Reactions
 - 04 Community announcements
 - 05 Following
 - 06 Activity
 - 07 Notifications
-

96. FOURTH DEVELOPMENT SPRINT

- 01 Conversations
 - 02 Messages
 - 03 Blocks
 - 04 Message requests
 - 05 Accountability partners
-

97. FIFTH DEVELOPMENT SPRINT

- 01 Challenges
 - 02 Challenge participation
 - 03 Challenge progress
 - 04 Challenge completion
 - 05 Community challenge management
-

98. SIXTH DEVELOPMENT SPRINT

- 01 Reports
 - 02 Moderation
 - 03 Moderator roles
 - 04 Admin dashboard
 - 05 Audit logs
 - 06 Rate limiting
 - 07 Security audit
-

99. SEVENTH DEVELOPMENT SPRINT

```
01 PWA
02 Offline shell
03 Offline check-in queue
04 Data-saving improvements
05 Mobile performance
06 3G testing
07 Accessibility testing
08 Cross-browser testing
```

100. THEN BETA

Invite a controlled group.

Do not advertise the product nationally yet.

Measure:

```
signup → onboarding completion
onboarding → community join
community join → goal creation
goal creation → first check-in
first check-in → second check-in
7-day retention
30-day retention
```

The most important question is:

Do people actually come back because the platform helps them stay accountable?

101. WHAT NOT TO DO

Do not let an AI coding agent:

```
invent database architecture repeatedly
rewrite the stack
create duplicate components
```



```
modify production schema casually
skip RLS
skip tests
hardcode secrets
install unnecessary packages
build features without specifications
replace working code unnecessarily
```

Do not let the project become:

```
AI generates code
→ another AI changes it
→ another AI changes it
→ nobody knows why anything exists
```

The documentation and repository architecture prevent this.

102. THE ACTUAL DEVELOPMENT LOOP

For every feature:

```
PRODUCT SPEC
  ↓
ARCHITECTURE
  ↓
DATABASE DESIGN
  ↓
SECURITY DESIGN
  ↓
IMPLEMENTATION
  ↓
UNIT TESTS
  ↓
INTEGRATION TESTS
  ↓
UI TEST
  ↓
SECURITY REVIEW
  ↓
CODE REVIEW
  ↓
DOCUMENTATION
```

↓
COMMIT
↓
PROJECT STATUS UPDATE

Repeat.

103. FINAL DEVELOPER COMMAND

If handing this repository to a senior developer or AI coding agent, the working instruction is:

Do not assume requirements that are not documented. Inspect the existing repository before making architectural decisions. Follow the feature specifications, database migrations, design system and security model. Reuse existing components and patterns. Before implementing a feature, identify dependencies and security implications. After implementing it, test the complete user flow, review authorization and edge cases, update documentation, and update project status. Preserve architectural consistency across the project so another developer can continue without relying on your memory.

104. WHAT THIS DOCUMENT IS

This document is the **engineering contract** for the product.

It is not a list of ideas.

It defines:

WHAT exists
WHO can use it
WHAT users can do
WHAT developers build
WHAT data exists
HOW systems interact
WHAT security is required
WHAT the frontend looks like
WHAT gets tested
WHEN something is considered finished
HOW another developer takes over

The remaining work is to turn each feature above into individual `FEATURE-XXX` specifications containing exact database schemas, TypeScript interfaces, server-action/API contracts, RLS policies, screen wireframes, component trees, validation rules and acceptance tests.

That is the level at which a senior developer can move from **specification** → **code** without having to redesign the product themselves.